CS458

SOFTWARE VERIFICATION AND VALIDATION

2025-2026 SPRING

PROJECT 1

ARES (AI-Driven Resilient & Evolutionary Systems)

Mehmet Hakan Yavuz
22002119

Türker Köken
22102331

Introduction **3**
System Preview **4**
Test Users: 4
File Structure: 4
App Screenshots: 5
System Architecture **10**
UML Diagrams: 10
i. Class Diagram 10
ii. Use-Case Diagram 10
iii. Sequence Diagram 11
iv. State Diagram 12
Advanced Test Scenarios: **13**
1. Dynamic ID Recovery 13
2. Multimodal Failure (Pop-up) 16
3. Cross-Browser CSS Breakage 18
4. Social Auth Handshake 20
5. Rate Limiting / Brute Force Simulation 23
6. Empty & Malformed Input Handling 27

Introduction

Login operations into a website seem simple. However, behind every login operation there is a complex set of decisions. There are some questions emerging when it comes to verification and validation such as: Is this user who they claim to be? Have they failed too many times? Is this behaviour suspicious to warrant a closer look? Focusing on these questions is what separates a secure authentication system from a vulnerable, straight-forward one.

In this project, ARES (AI-Driven Resilient & Evolutionary Systems), we aimed to design a web-based login platform generated to explore how LLM's (AI tools) can make both security and testing processes smarter and adaptable. ARES watches every login attempt and assigns a risk score based on factors like failed attempts, unfamiliar IP addresses, suspicious browser signatures, and unusual login times. If that risk score is high enough to pass a certain threshold an AI model is called to make a judgement for violation reasoning, eventually deciding if the user should be allowed, warned or suspended. Additionally, the automated tests built for ARES system are designed to fix themselves when things go wrong, rather than simply failing and having the developer take initiative.

The testing part is just as important as the application itself. Real web apps change constantly (buttons name change, popups might appear unexpectedly, visual styles (CSS) might break at certain points etc.), and a system that can adapt those changes is beneficial. In conventional testing scenarios any of these changes can cause an automated test to fail even when the application works without flaw. ARES addresses this by building a test environment that uses AI-assisted recovery before giving up.

Six test cases were generated to cover the most crucial aspects of the system; automatic recovery from broken elements, handling unexpected pop-ups, resilience to visual feature breakage, real login flows using Google and Facebook, brute-force attack detection, and secure handling of irrelevant inputs.

The goal of this project is to demonstrate that AI integration in both security and testing is a practical and beneficial approach.

System Preview

Test Users:

```
## Pre-seeded Test Users

| Name           | Email            | Password  |
|---------------|-----------------|-----------|
| Alice Johnson | alice@ares.test | Test1234! |
| Bob Smith     | bob@ares.test   | Secure99# |
```

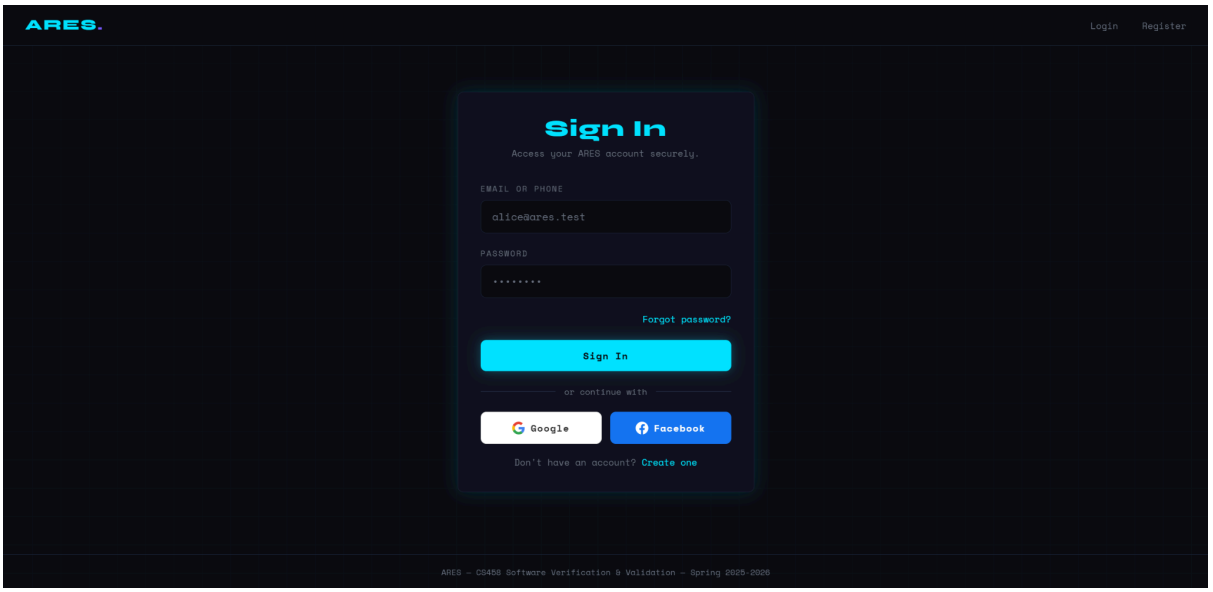
```
TEST_GOOGLE_EMAIL      = "ares.test.cs458@gmail.com"
TEST_GOOGLE_PASSWORD   = "123test456"
TEST_FACEBOOK_EMAIL    = "ares.test.cs458@gmail.com"
TEST_FACEBOOK_PASSWORD = "1234test5678"
```

File Structure:

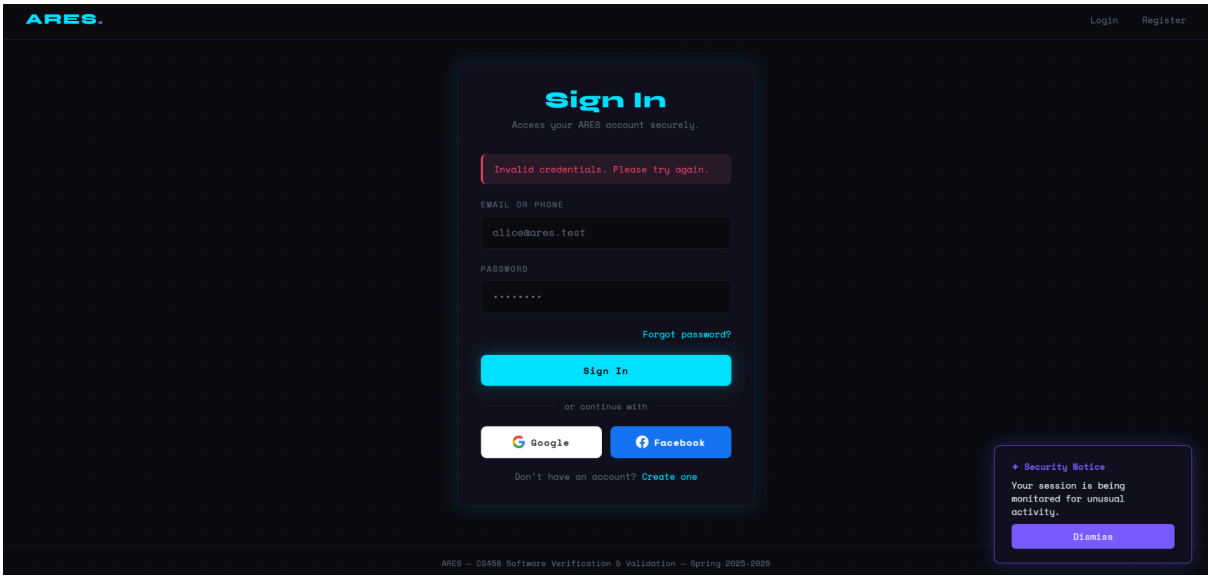
```
ares_auth/
├── app.py                  Flask application & routes
├── auth_store.py           In-memory user/session store
├── risk_engine.py          Risk scoring & LLM integration
├── oauth_providers.py      Google & Facebook OAuth helpers
├── self_healing_driver.py  Self-healing Selenium framework
├── tests.py                Full test suite (TC1-TC6 + RC1-RC4)
└── templates/
    ├── base.html           Base layout (dark theme, animated grid)
    ├── login.html          Login page with risk panel
    ├── dashboard.html      Post-login dashboard
    └── register.html       Registration page
```

App Screenshots:

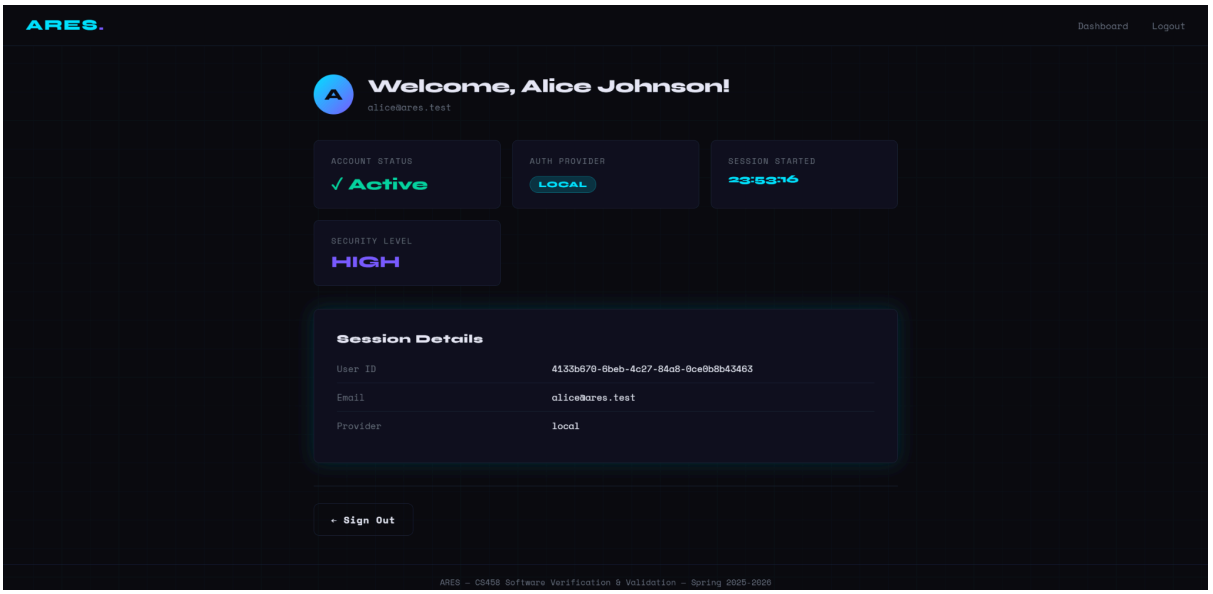
1. Default Sign-In Page



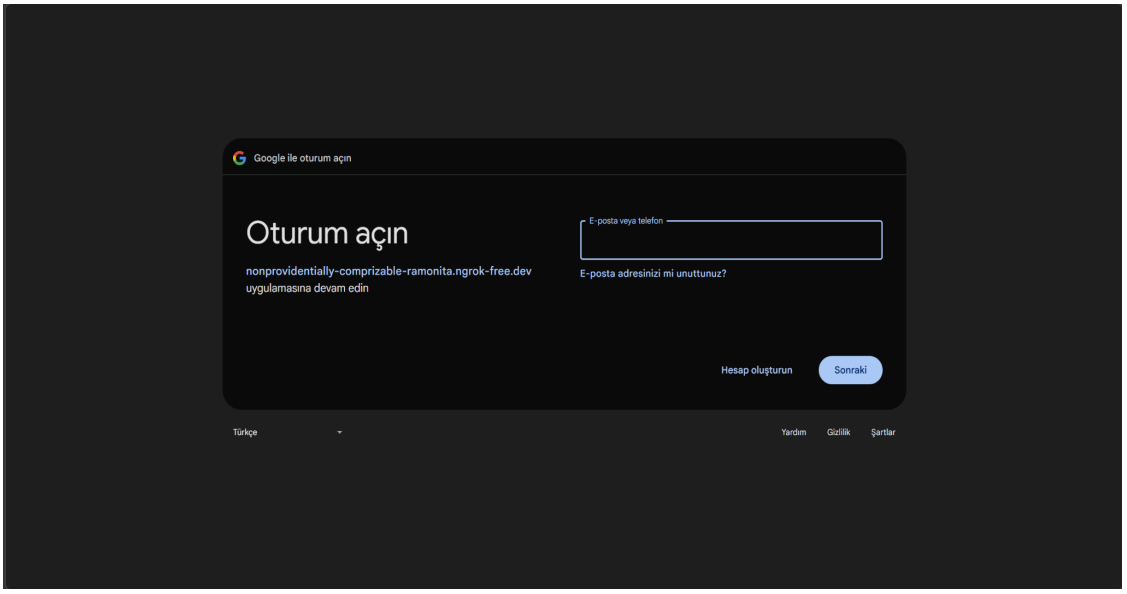
2. Invalid Credentials Error



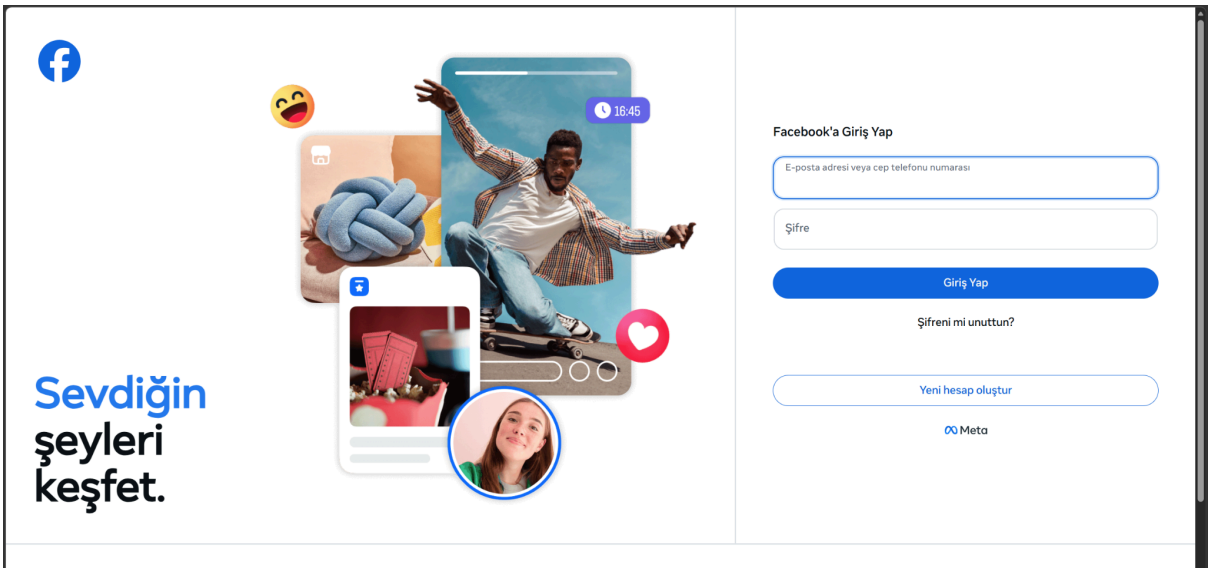
3. **Signed-In as Alice Johnson (ARES Account)**



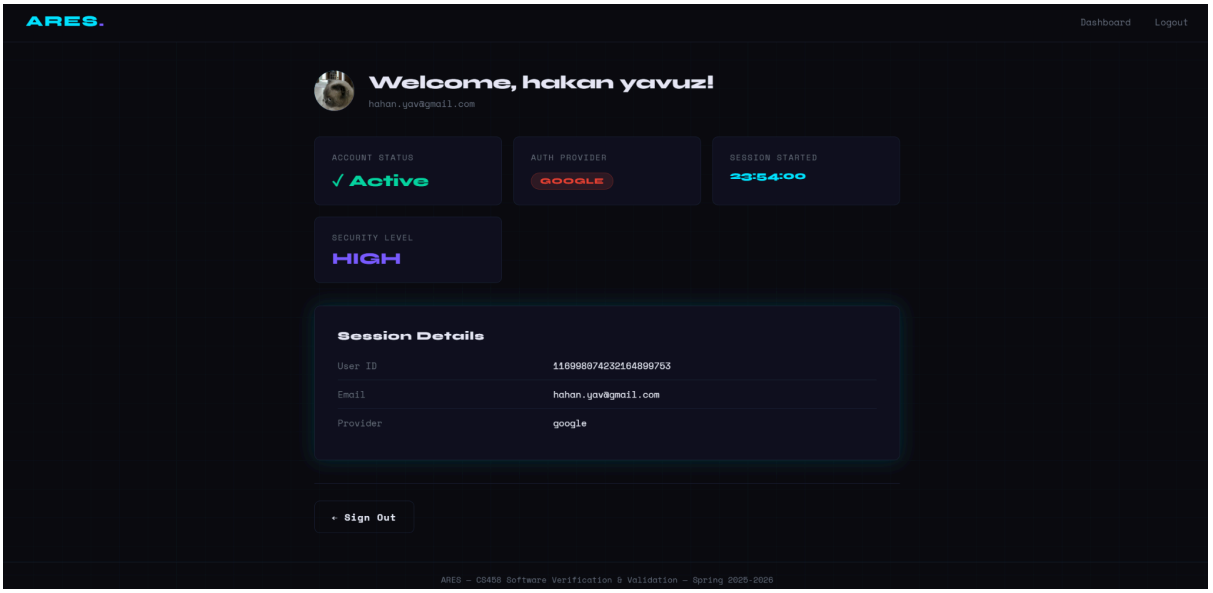
4. **Redirected to Google SIngle Sign-on**



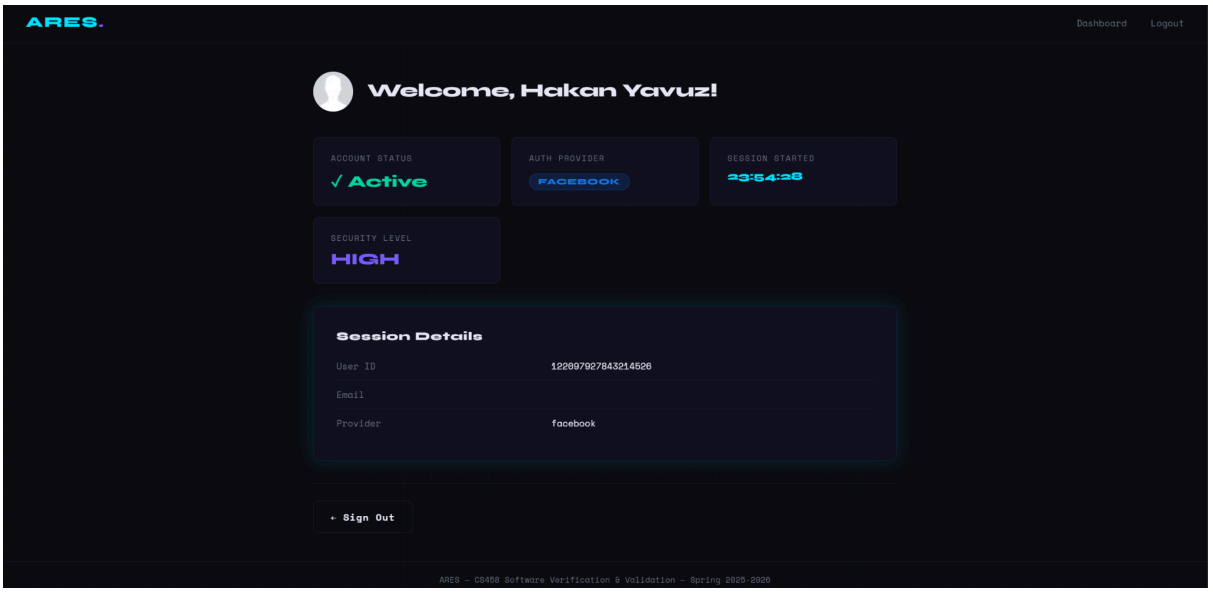
5. **Redirected to Facebook SIngle Sign-on**



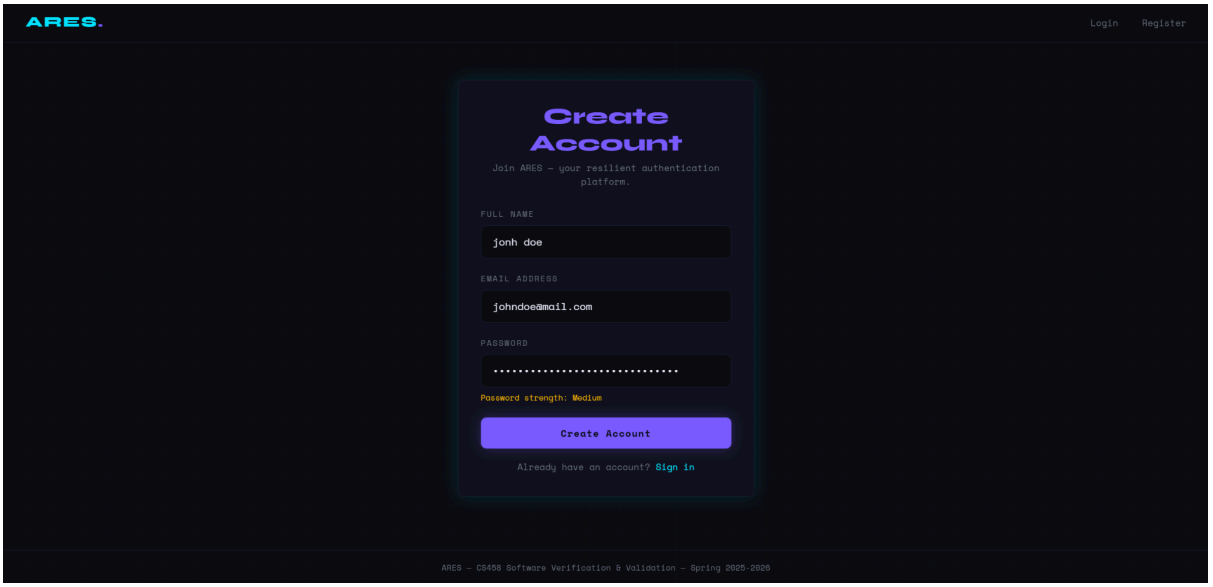
6. **Signed-In as Hakan Yavuz (Google Account)**



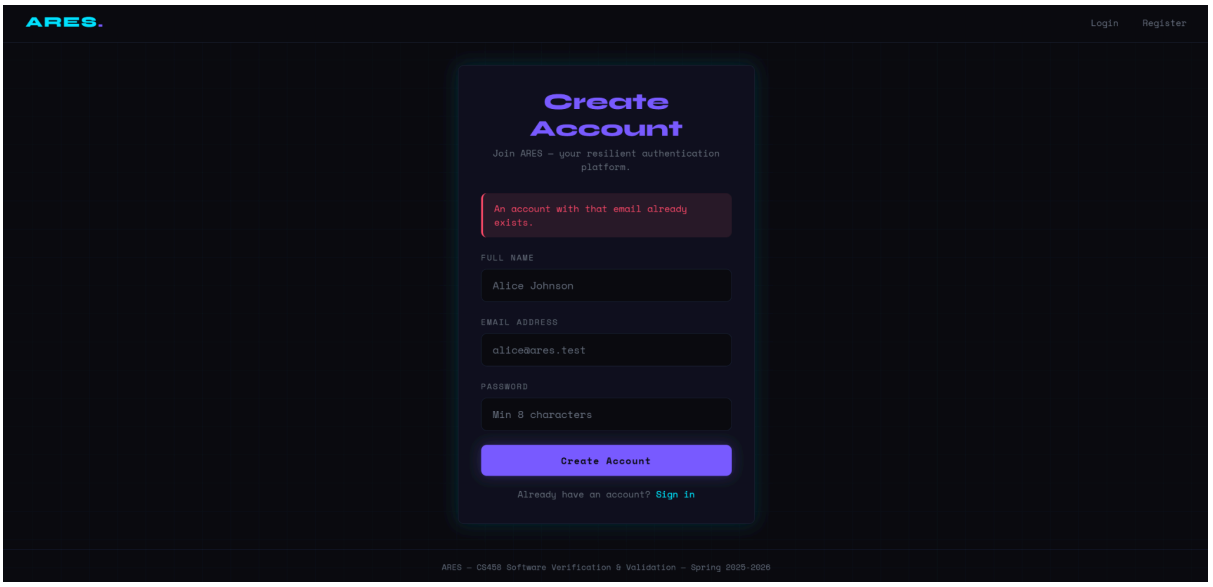
7. Signed-In as Hakan Yavuz (Facebook Account)



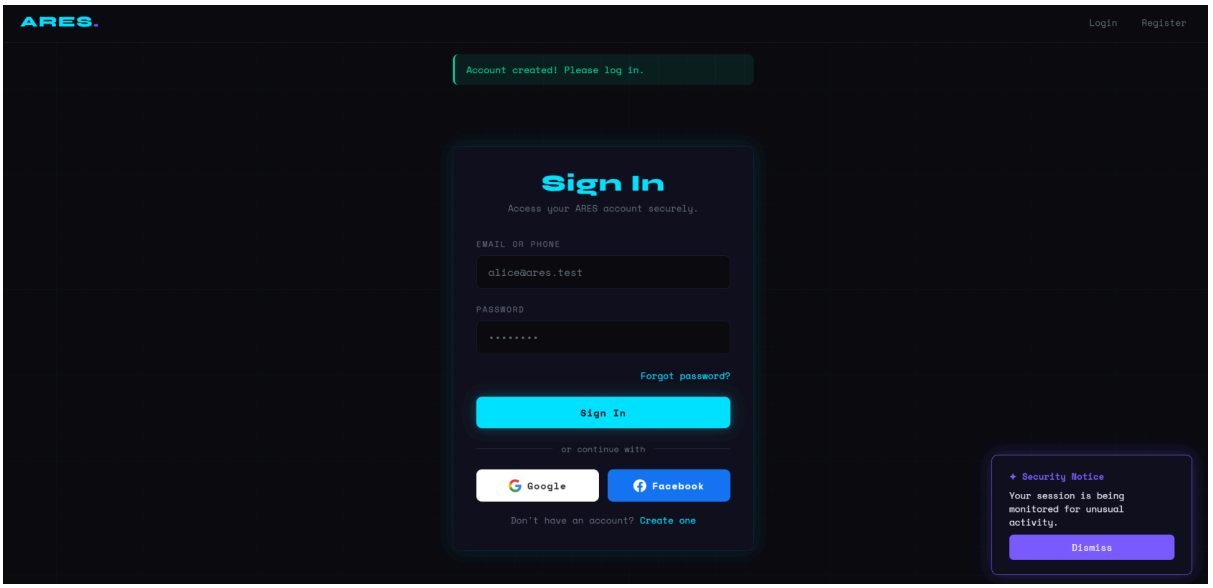
8. Account Creation Page



9. E-mail Already in Use Error during Account Creation



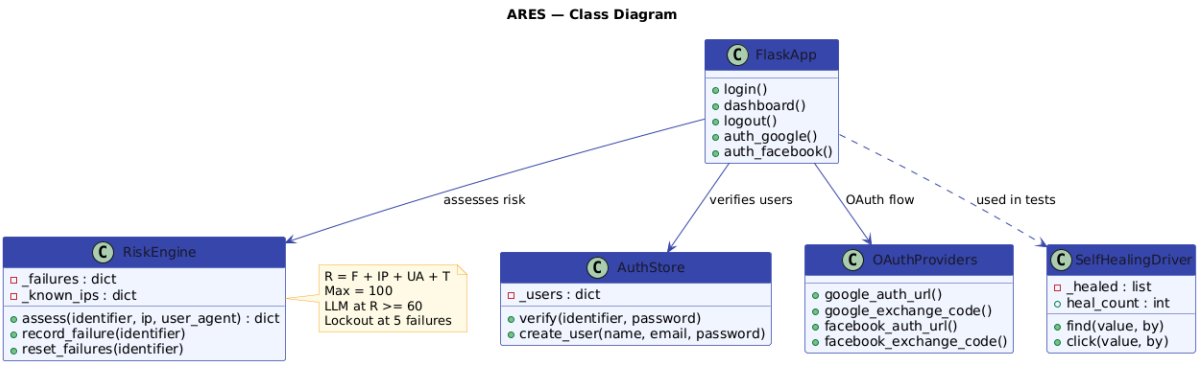
10. Account Creation Succesful Notification



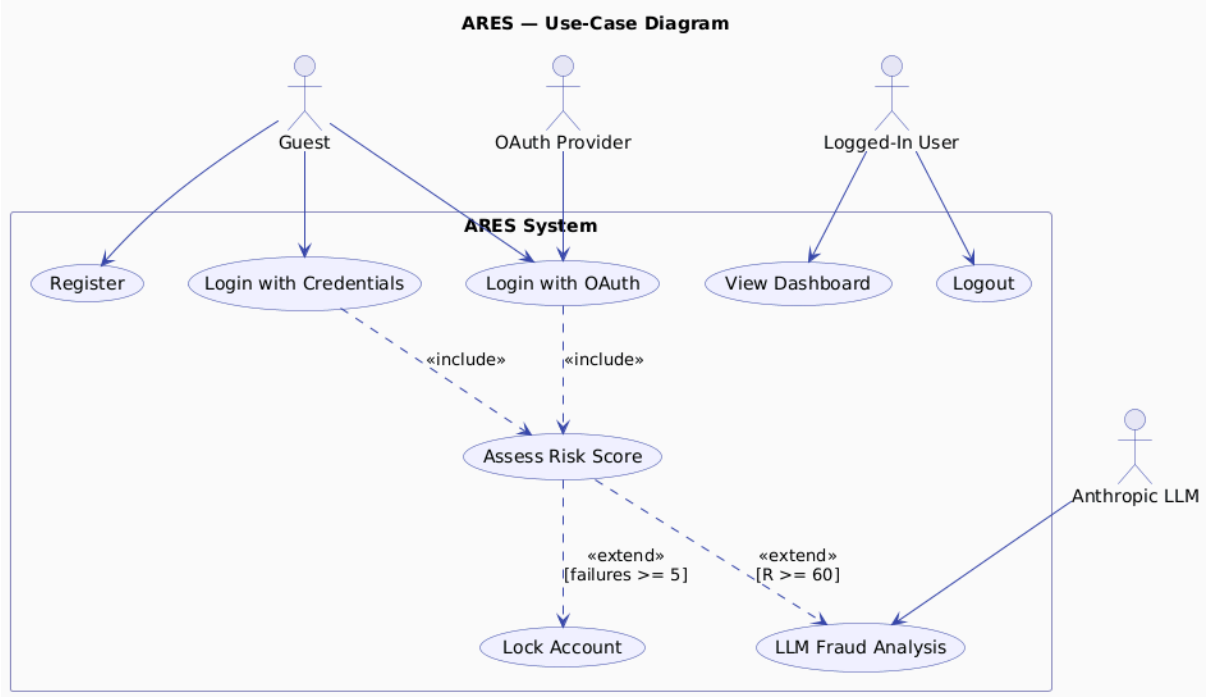
System Architecture

UML Diagrams:

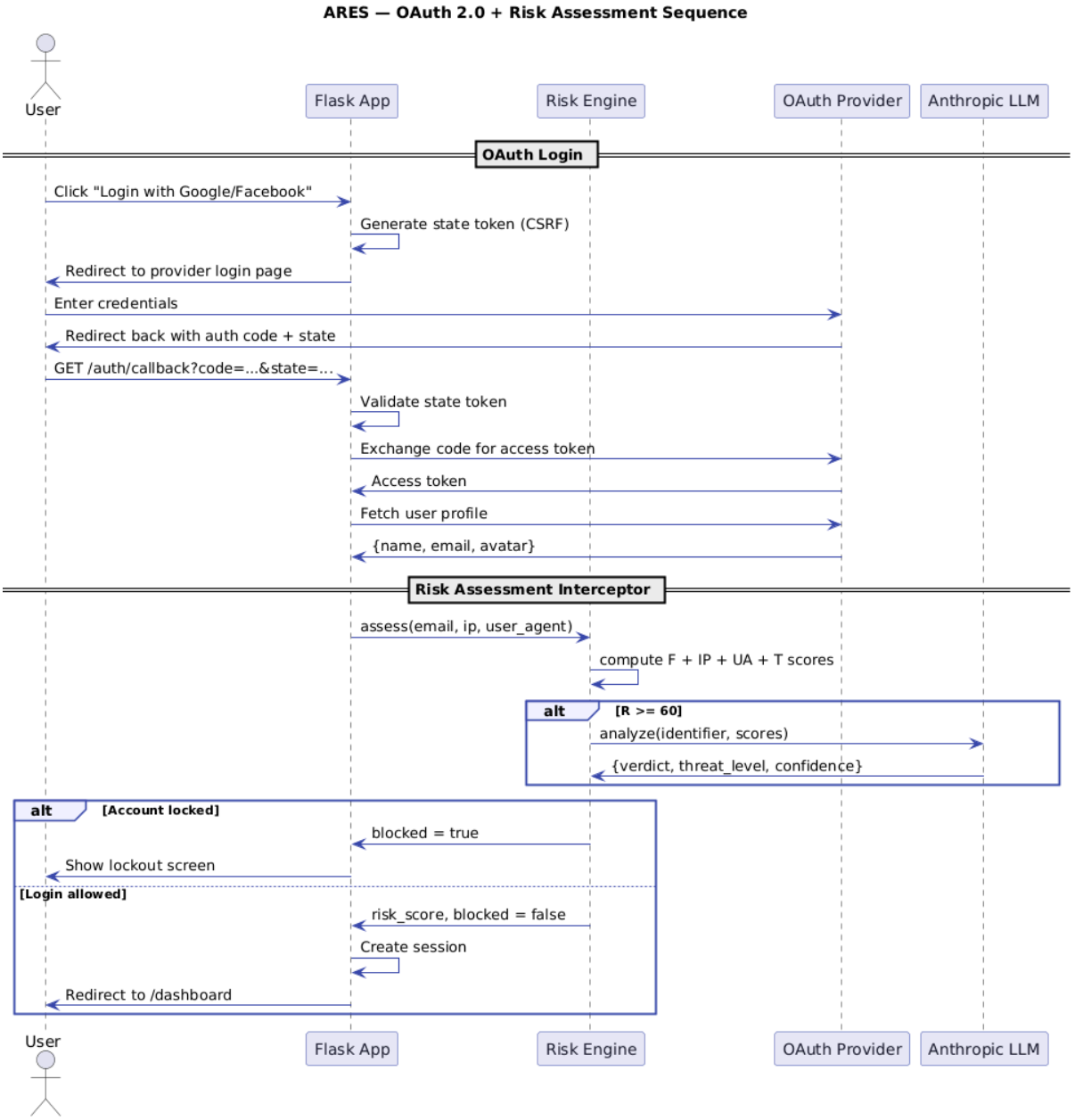
i. Class Diagram



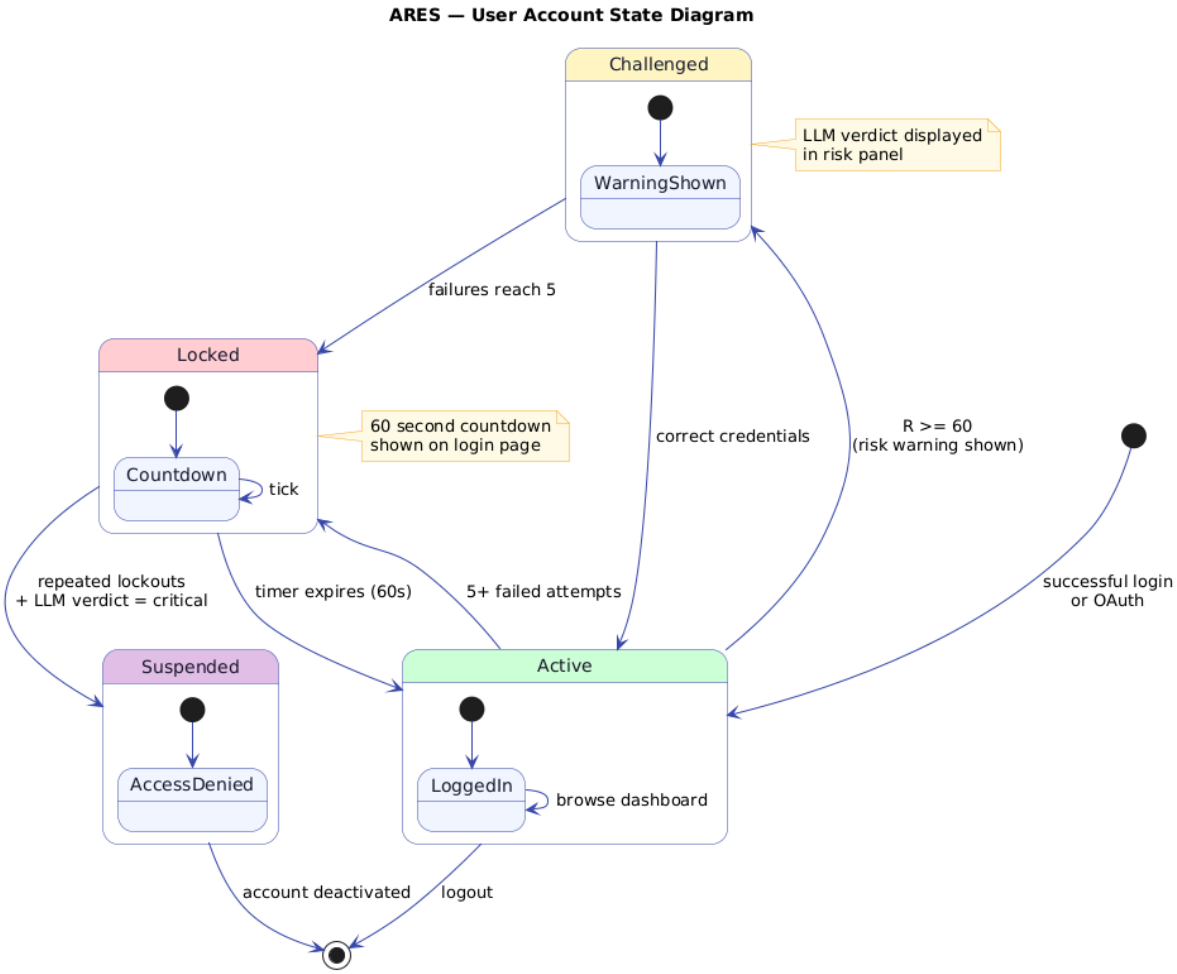
ii. Use-Case Diagram



iii. Sequence Diagram



iv. State Diagram



Advanced Test Scenarios:

1. Dynamic ID Recovery

Tests the self-healing framework's ability to recover from a broken element selector. The login button's ID is renamed at runtime, simulating front-end frameworks that regenerate element IDs dynamically. The framework detects the failure, sends the DOM context to the Claude API, and locates the correct element under its new identifier, completing the login without any manual intervention.

Runtime ID Mutation (tests.py)

```
# Mutate the login button ID at runtime
        driver.execute_script(
            "document.getElementById('login-btn').id =
'login-submit-2025';"
        )
```

Self-Healing Entry Point (self_healing_driver.py)

```
def _heal(self, original_by: str, original_value: str) ->
Optional[WebElement]:
        """Extract DOM context, call LLM, apply healed selector."""
        dom_snippet = self._extract_dom_context(original_by,
original_value)
        url         = self.driver.current_url

        healed_by, healed_value, reasoning = self._llm_repair(
            original_by, original_value, dom_snippet, url
        )

        if not healed_value:
            print(f"  [HEAL] LLM could not repair selector:
{original_by}='{original_value}'")
            return None

        print(f"  [HEAL] Healed: {original_by}='{original_value}' →
{healed_by}='{healed_value}'")
```

When *find("login-btn")* raises *NoSuchElementException*, the code *calls _heal()*. It first captures a relevant DOM window around where the element was expected, then passes that context to the LLM repair engine. The result is a new selector that points to the same element under its new ID.

LLM System Prompt for Selector Repair (self_healing_driver.py)

```
 system_prompt = (
            "You are a Selenium self-healing engine. "
            "You receive a broken CSS/XPath selector and a DOM snippet.
"
            "You must respond ONLY with a JSON object — no markdown, no
explanation — with these exact fields:\n"
            '{"strategy":
"css"|"xpath"|"id"|"name"|"class_name"|"tag_name", '
            '"selector": "<valid selector string>", '
            '"reasoning": "<one sentence>"}\n'
            "The selector must target the element the broken one was
looking for. "
            "Never invent attributes that don't exist in the DOM. "
            "Prefer CSS selectors unless XPath is clearly better. "
            "If you cannot find a valid selector, return: "
            '{"strategy": "css", "selector": "", "reasoning": "element
not present"}'
        )
```

This is the LLM prompt sent to Claude. It asks the model to return JSON with precise fields that we use. The "never invent attributes" instruction is intended to reduce and prevent hallucination. The DOM snippet from the web page is also sent to the model for visual analysis.

Heuristic Scoring (self_healing_driver.py)

```
 for el in candidates:
                score = 0
                el_id   = (el.get_attribute("id")    or "").lower()
                el_name = (el.get_attribute("name")  or "").lower()
                el_cls  = (el.get_attribute("class") or "").lower()
                el_text = (el.text or "").lower()
                el_type = (el.get_attribute("type")  or "").lower()
                el_tag  = el.tag_name.lower()
                el_href = (el.get_attribute("href")  or "").lower()
```

```
                for tok in tokens:
                    if tok and tok in el_id:   score += 40
                    if tok and tok in el_name: score += 30
                    if tok and tok in el_cls:  score += 20
                    if tok and tok in el_text: score += 15

                # Strong bonus for actual submit/button on login
targets
                if is_login_target and el_type == "submit":
                    score += 50
                if is_login_target and el_tag == "button":
                    score += 30

                # Heavy penalty: social auth links are NOT the login
submit button
                if is_login_target and any(
                    s in el_id for s in ["google", "facebook",
"social", "twitter"]
                ):
                    score -= 80
                if is_login_target and any(
                    s in el_href for s in ["/auth/google",
"/auth/facebook"]
                ):
                    score -= 80
```

The framework also includes token-based scoring. It tokenises the original selector value (*login-btn* > *{login, btn}*), scores every interactive element on the page, and applies penalties to social authentication buttons to prevent false matches. This method improves recovery even without API access.

Heal Log Output (self_healing_driver.py)

```
self._heal_log.append(HealedElement(
            original_by=original_by,
            original_value=original_value,
            healed_by=healed_by,
            healed_value=healed_value,
            url=url,
            timestamp=datetime.utcnow().isoformat(),
            llm_reasoning=reasoning
        ))
```

Every healing event is recorded to heal_log.json. This json includes original selector, replacement selector, page URL, timestamp, and the LLM's one-sentence reasoning. This log can be reviewed after a test run to understand which changes caused failures.

2. Multimodal Failure (Pop-up)

Tests the framework's ability to handle UI obstructions. A full-screen pop-up is injected over the login form, blocking interaction with the Google OAuth button. The framework detects the “ElementClickInterceptedException”, identifies and dismisses the popup, then retries the click, verifying that transient UI obstructions do not cause test failures.

Pop-up Injection (tests.py)

```
# Force-inject a popup that covers the Google button
        driver.execute_script("""
            const popup = document.createElement('div');
            popup.id = 'promo-popup';
            popup.style.cssText = `
                position: fixed;
                top: 0; left: 0; right: 0; bottom: 0;
                background: rgba(0,0,0,0.85);
                z-index: 99999;
                display: flex;
                align-items: center;
                justify-content: center;
            `;
            popup.innerHTML = `
                <div style="background:#111320; border:1px solid
#7b5cff;
                            border-radius:8px; padding:32px;
text-align:center;">
                    <h3 style="color:#7b5cff;
font-family:sans-serif;">Security Notice</h3>
                    <p style="color:#e8eaf6; font-family:sans-serif;
margin:12px 0;">
                        Suspicious activity detected. Please confirm
you are human.
                    </p>
                    <button id="close-popup-btn"
```

```
                        style="background:#7b5cff; color:#fff;
border:none;
                                padding:10px 24px; border-radius:6px;
                                cursor:pointer;
font-family:sans-serif;">
                        I understand — Dismiss
                    </button>
                </div>
            `;
            document.body.appendChild(popup);
        """)
```

This block injects a full-screen modal with z-index: 99999 directly over the entire UI. This simulates an unexpected pop-up that blocks element interaction. If Selenium tries to use *.click()* on elements beneath the pop-up, it will throw an *ElementClickInterceptedException*.

<u>Interception Detection and Recovery (tests.py)</u>

```
try:
            google_btn.click()
            info("Google button clicked directly (popup may have
auto-closed)")
        except (ElementClickInterceptedException, Exception):
            info("Click intercepted by popup — closing popup via
self-healing logic")

            # Step 1: find and JS-click the dismiss button (avoids
popup intercepting it)
            close_btn = shd.find("close-popup-btn", By.ID)
            if close_btn:
                driver.execute_script("arguments[0].click();",
close_btn)
                popup_closed = True
                ok("Popup dismiss button clicked via JS")
            else:
                driver.execute_script(
                    "var p = document.getElementById('promo-popup');
if(p) p.remove();"
                )
                popup_closed = True
                ok("Popup force-removed via JS")
```

```
            # Step 2: wait until popup is fully invisible/removed from
DOM
            try:
                WebDriverWait(driver, 5).until(
                    EC.invisibility_of_element_located((By.ID,
"promo-popup"))
                )
                ok("Popup confirmed gone from DOM")
            except TimeoutException:
                driver.execute_script(
                    "var p = document.getElementById('promo-popup');
if(p) p.remove();"
                )
                ok("Popup force-removed after wait timeout")

            # Step 3: JS-click the Google button — avoids any lingering
interception
            google_btn = shd.find("google-login-btn", By.ID)
            driver.execute_script("arguments[0].click();", google_btn)
            ok("Google button clicked via JS after popup dismissed")
```

The framework catches *ElementClickInterceptedException*, identifies the blocking element by ID, dismisses it by bypassing the z-index stack, and waits for full DOM removal, then fetches and clicks the original target.

3. Cross-Browser CSS Breakage

Tests whether the application remains functionally operable even when its visual rendering is completely broken. CSS injection makes inputs invisible, buttons transparent, and layout collapsed, to simulate browser rendering inconsistencies. The test verifies that all form elements still exist in the DOM and that login can be completed programmatically despite broken styles.

CSS Injection (tests.py)

```
# Inject intentionally broken CSS
        driver.execute_script("""
            const style = document.createElement('style');
            style.id = 'broken-css';
            style.textContent = `
                /* Simulate catastrophic CSS breakage */
```

```
                * { font-family: 'NonExistentFont123' !important; }
                .card { display: table !important; width: 50%
!important; }
                .form-control { background: black !important; color:
black !important; }
                .btn { opacity: 0.1 !important; transform: skewX(45deg)
!important; }
                body { zoom: 0.5; }
            `;
            document.head.appendChild(style);
        """)
```

This block simulates a CSS failure or its misconfiguration. *color: black !important* on black backgrounds makes inputs invisible to human eyes. *opacity: 0.1* makes buttons nearly transparent. *zoom: 0.5* collapses the layout. The test verifies whether DOM elements remain present and functional after these altercations.

DOM Presence Verification + JS Click (tests.py)

```
 # Verify elements still exist in DOM despite broken styles
        elements_to_check = [
            ("identifier",   By.ID,   "Email/phone input"),
            ("password",     By.ID,   "Password input"),
            ("login-btn",    By.ID,   "Login button"),
            ("login-form",   By.ID,   "Login form"),
        ]

        all_found = True
        for selector, by, label in elements_to_check:
            el = shd.find(selector, by)
            if el:
                ok(f"DOM element present: {label}")
            else:
                fail(f"DOM element MISSING: {label}")
                all_found = False

        if not all_found:
            record("TC3", False, "One or more elements missing despite
CSS breakage")
            return

        # Attempt login with broken CSS still active
```

```
        shd.type_into("identifier", VALID_EMAIL)
        shd.type_into("password",   VALID_PASSWORD)

        # Use JS click to bypass opacity:0.1 CSS visibility issue

driver.execute_script("document.getElementById('login-btn').click();")
        info("Submitted login form via JS (bypassing CSS opacity)")
```

This test verifies that DOM still allows Selenium to navigate through the website despite broken visual rendering. Even when the UI is visually broken, automated tests can still drive the application correctly thanks to DOM snippets.

4. Social Auth Handshake

Tests the complete OAuth 2.0 authentication flow for both Google and Facebook. Selenium navigates to the provider login page, types real credentials using a Chrome instance, and follows the redirect chain back through the ngrok tunnel to the ARES dashboard. The test verifies that the CSRF state token is validated, the authorization code is exchanged correctly, and the user profile is displayed on the dashboard.

SRF State Token and Redirect (app.py)

```
@app.route("/auth/google/callback")
def auth_google_callback():
    state = request.args.get("state", "")
    if not state or state != session.pop("oauth_state", None):
        flash("OAuth state mismatch — possible CSRF attack.", "danger")
        return redirect(url_for("login"))

    error = request.args.get("error")
    if error:
        flash(f"Google login cancelled: {error}", "warn")
        return redirect(url_for("login"))
```

A random state token is generated per login attempt and stored in the session. When Google redirects back with a code, the state is validated before any token exchange occurs. If a mismatch occurs, the session stops and issues a warning against authorization code interception attacks (CSRF attack etc.).

Authorization URL Construction (oauth_providers.py)

```
def google_auth_url(redirect_uri: str, state: str) -> str:
    """Build the Google authorization URL to redirect the user to."""
    client_id = os.environ.get("GOOGLE_CLIENT_ID", "")
    if not client_id:
        raise OAuthConfigError(
            "GOOGLE_CLIENT_ID is not set. "
            "Get credentials at console.cloud.google.com"
        )
    params = {
        "client_id":     client_id,
        "redirect_uri":  redirect_uri,
        "response_type": "code",
        "scope":         GOOGLE_SCOPES,
        "state":         state,
        "access_type":   "online",
        "prompt":        "select_account",   # always show account
picker
    }
    return GOOGLE_AUTH_URL + "?" + urllib.parse.urlencode(params)
```

This block constructs a URL for OAuth 2.0 authorization. *"response_type": "code"* is the authorization code.

Code Exchange and Profile Fetch (oauth_providers.py)

```
# Step 1: Exchange code for access token
    token_data = urllib.parse.urlencode({
        "code":          code,
        "client_id":     client_id,
        "client_secret": client_secret,
        "redirect_uri":  redirect_uri,
        "grant_type":    "authorization_code",
    }).encode("utf-8")

    token_req = urllib.request.Request(
        GOOGLE_TOKEN_URL,
        data=token_data,
        headers={"Content-Type": "application/x-www-form-urlencoded"},
        method="POST"
```

```
    )
    try:
        with urllib.request.urlopen(token_req, timeout=10) as resp:
            tokens = json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        detail = e.read().decode("utf-8", errors="replace")
        raise OAuthCallbackError(f"Google token exchange failed:
{detail}") from e

    access_token = tokens.get("access_token", "")
    if not access_token:
        raise OAuthCallbackError("Google returned no access token.")

    # Step 2: Fetch user info
    userinfo_req = urllib.request.Request(
        GOOGLE_USERINFO_URL,
        headers={"Authorization": f"Bearer {access_token}"}
    )
    try:
        with urllib.request.urlopen(userinfo_req, timeout=10) as resp:
            userinfo = json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        detail = e.read().decode("utf-8", errors="replace")
        raise OAuthCallbackError(f"Google userinfo fetch failed:
{detail}") from e

    return {
        "id":           userinfo.get("sub", ""),
        "name":         userinfo.get("name", "Google User"),
        "email":        userinfo.get("email", ""),
        "avatar":       userinfo.get("picture"),
        "provider":     "google",
        "access_token": access_token,
    }
```

The authorization code is exchanged for an access token. This exchange is not exposed to the browser. Then the access token is used to call the *userinfo* endpoint. The system uses the Google user ID as the system's user identifier.

Selenium Live Credential Typing (tests.py)

```
python# Step 1: Enter email character by character
email_field = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.CSS_SELECTOR,
"input[type=email]"))
)
slow_type(email_field, google_email, delay=0.07)
next_btn.click()

# Step 2: Enter password
pwd_field = WebDriverWait(driver, 10).until(
    EC.element_to_be_clickable((By.CSS_SELECTOR,
"input[type=password]"))
)
slow_type(pwd_field, google_password, delay=0.07)
pwd_next.click()

# Step 3: Wait for OAuth callback → dashboard
WebDriverWait(driver, 30).until(
    lambda d: "/dashboard" in d.current_url
)
```

slow_type helper allows Selenium to type slowly to mimic human input timing. The 30-second wait time accounts for network latency and provides ease of observability during the test.

5. Rate Limiting / Brute Force Simulation

Tests the risk engine's lockout mechanism under simulated brute-force conditions. Six consecutive login attempts are made with incorrect passwords. The test verifies that the risk score escalates with consequent failures, that the LLM fraud analysis is triggered when the score reaches 60, and that the account is locked after this point, not allowing for another log in attempt in the next 60 seconds.

Failure Loop (tests.py)

```
pythonfor attempt in range(1, 7):
    shd.type_into("identifier", VALID_EMAIL)
    shd.type_into("password",   f"WrongPassword{attempt}!")
    shd.click("login-btn", By.ID)

    error_el = driver.find_element(By.ID, "login-error")
    if "locked" in error_el.text.lower():
```

```
        lockout_detected = True
        break
```

Six consecutive failed attempts with wrong passwords are submitted in a loop. Each uses a unique password to simulate a real-life scenario and prevent any caching effect. After each attempt the error element is inspected by the framework. The test is successfully finished when "locked" appears in the message.

Risk Score Calculation (risk_engine.py)

```
## Risk Scoring Formula

```
R = F_score + IP_score + UA_score + T_score   (max 100)
```

Component	Max	Trigger
F_score	40	Failed login attempts
IP_score	30	Unknown / new IP address
UA_score	20	Suspicious User-Agent string
T_score	10	Unusual login time

LLM fraud analysis (via Anthropic API) is triggered when R >= 60.

```

```
# F_score
        f_score = next(pts for (thresh, pts) in FAILURE_SCORE_TABLE
                       if failures >= thresh)
        breakdown["F_score"] = f_score
        if f_score > 0:
            reasons.append(f"Failed login attempts: {failures}
(+{f_score} pts)")

        # IP_score
        ip_score = 0
        if identifier and ip not in self._known_ips.get(identifier,
[]):
            ip_score = IP_NEW_SCORE
```

```
            reasons.append(f"Login from unrecognised IP {ip}
(+{ip_score} pts)")
        breakdown["IP_score"] = ip_score

        # UA_score
        ua_score = 0
        ua_hit   = next((kw for kw in SUSPICIOUS_UA_KEYWORDS
                         if kw in user_agent.lower()), None)
        if ua_hit:
            ua_score = UA_SUSPICIOUS
            reasons.append(f"Automation tool in user-agent: '{ua_hit}'
(+{ua_score} pts)")
        breakdown["UA_score"] = ua_score

        # T_score
        t_score  = 0
        utc_hour = datetime.now(timezone.utc).hour
        if NIGHT_HOUR_START <= utc_hour < NIGHT_HOUR_END:
            t_score = TIME_ANOMALY
            reasons.append(f"Unusual login hour {utc_hour:02d}:xx UTC
(+{t_score} pts)")
        breakdown["T_score"] = t_score

        total = min(f_score + ip_score + ua_score + t_score, 100)

        # LLM analysis — only when threshold is exceeded
        llm_analysis = None
        llm_error    = None
        if total >= LLM_TRIGGER_SCORE:
            try:
                llm_analysis = self._llm_analyze(
                    identifier, ip, user_agent,
                    failures, reasons, breakdown, total
                )
            except LLMUnavailableError as e:
                llm_error = str(e)
                print(f"  [RISK] LLM unavailable — analysis skipped:
{e}")

        return {
            "risk_score":      total,
            "blocked":         False,
            "lockout_remaining": 0,
```

```
            "score_breakdown": breakdown,
            "reasons":         reasons,
            "llm_analysis":    llm_analysis,
            "llm_error":       llm_error }
```

We decided that our system had to look for four distinct risk factors: Failure (wrong credentials), IP address, User Agent activity, and Log in Time. These suspicious activities shared a scoring system, where we calculate the risk score of the user's activities.
When this score reaches 60 or higher, the system triggers an LLM-based fraud analysis. Firstly, the simplest risk factor is failure in credentials. It indicates another user might be attempting to log in to this account and they're brute forcing passwords, however this failure could also be caused by the owner of the account simply forgetting their password. For this risk factor, we issued a step table (1-2 attempts → +15, 3-4 → +35, 5-9 → +55, 10+ → +75). The system only raises the risk score if there were too many attempts, while being forgiving towards the first few attempts. Secondly, for IP address change, we look at known IPs of the user, if the user's IP does not match with the IPs they've used in the past, we raise the risk score by 20. We decided 20 because if the user is logging in from unusual places, they likely log in very often (i.e. not just during work or from home, but also from other places which often indicates they use the account quite frequently) and would remember their credentials correctly. Raising the risk score would mean fewer attempts in failed log-ins, but this user will rarely put in wrong credentials, which means raising the score by 20 is not as punishing for the owner of the account. However it is still an increase in risk score and will be punishing if failed log in attempts occur (which would likely be a fraud scenario) and will trigger 'locked' state after just three failed log in attempts. Thirdly, we inspected user agent activity. We check if the user agent string contains a known automation/attack tool keyword. Checked keywords are : “sqlmap, nikto, nmap, masscan, zgrab, python-requests, go-http-client, curl/, wget/, libwww, scrapy, hydra, burpsuite”. If the user-agent tries to access the login form through a browser will essentially never have one of these keywords. The detection of automated activity will be penalized by 30 points on risk score. Lastly, we accounted for time of log-in. An unusual log-in time does not indicate a direct risk but it should still raise suspicion. For this factor we raise the risk score by 10 if the user is attempting to log in between 00.00-05.00 local time.

<u>LLM System Prompt for Fraud Analysis (risk_engine.py)</u>

```
pythonsystem_prompt = (
    "You are ARES, an AI-powered security fraud-analysis engine. "
    "You receive a structured login event log and must decide
whether "
    "to ALLOW, CHALLENGE, or BLOCK the attempt.\n\n"
    "Risk Score Formula:\n"
    "  R = F_score + IP_score + UA_score + T_score  (max 100)\n"
    "  F_score : 1-2 failures=15, 3-4=35, 5-9=55, 10+=75\n"
    "  IP_score: unknown IP=+20\n"
    "  UA_score: automation tool in UA=+30\n"
```

```
    "  T_score : login at 00:00-05:00 UTC=+10\n"
    "  LLM triggered when R >= 60\n\n"
    'Respond ONLY with JSON:
{"verdict":"ALLOW"|"CHALLENGE"|"BLOCK",'

'"confidence":<0-100>,"summary":"...","recommended_action":"...",'
    '"threat_level":"LOW"|"MEDIUM"|"HIGH"|"CRITICAL"}'
)
```

The LLM prompt explains the risk scoring system so the LLM understands the context of each number in the event log. We ask for the output in JSON form so that it is easily machine-readable. The *ALLOW/CHALLENGE/BLOCK* indicates the verdict of the LLM in the risk panel.

Lockout Verdict (risk_engine.py)

```
pythonif state["locked_until"] and now < state["locked_until"]:
    remaining = int(state["locked_until"] - now)
    return {
        "risk_score": 100,
        "blocked":    True,
        "lockout_remaining": remaining,
        ...
    }
```

When the LLM decides to *BLOCK*, lockout is initiated. This period is triggered when risk score is over 60, and it persists for 60 seconds. The remaining seconds are displayed in the UI for the countdown timer. The record_failure method sets *locked_until = now + LOCKOUT_DURATION*.

6. Empty & Malformed Input Handling

Tests the robustness of the login form against seven categories of malicious and malformed input: empty fields, SQL injection strings, XSS payloads, null bytes, 512-character overlong strings, Unicode right-to-left text, and whitespace-only values. Each payload must be rejected without crashing the server, and the XSS payload must not be reflected back in the error message.

Payload Table (tests.py)

```
pythonPAYLOADS = [
    ("Empty fields",    "",                          ""),
    ("SQL injection",   "' OR '1'='1",               "' OR
'1'='1"),
    ("XSS payload",     "<script>alert(1)</script>", "xss"),
```

```
    ("Null bytes",      "admin\x00@test.com",        "pass"),
    ("Overlong input",  "A" * 512,                   "B" * 512),
    ("Unicode / RTL",   "ادمين@test.com",           "كلمةالسر"),
    ("Whitespace only", "   ",                       "   "),
]
```

The system covers seven different attack attempts. Each tuple consists of label, identifier, and password. The SQL injection tests whether the backend queries are parameterised. The XSS payload tests output encoding. Null bytes test binary safety. The 512 character-long strings test that no buffer overflow or unbounded memory allocation occurs.

Rejection and Crash Detection (tests.py)

```
pythonif "/dashboard" in current:
    fail(f"{label}: UNEXPECTED LOGIN — payload accepted as
credentials!")
elif "500" in page or "Internal Server Error" in page or
"Traceback" in page:
    fail(f"{label}: Server crashed with 500 error!")
else:
    ok(f"{label}: Rejected safely, stayed on login page")
```

The framework features three distinct failure modes. The test checks if the user reaches */dashboard* which means the payload was mistakenly accepted as valid credentials, resulting in a security failure. A 500 response or Python traceback means the server crashed, also a failure. Only staying on */login* with the user receiving an error message counts as a pass.

XSS Reflection Check (tests.py)

```
pythonif label == "XSS payload":
    error_el = driver.find_element(By.ID, "login-error")
    error_html = error_el.get_attribute("innerHTML")
    if "<script>" in error_html:
        fail("XSS: Script tag reflected unescaped in error
message!")
    else:
        ok("XSS: Script tag safely escaped in error message ✓")
```

The test inspects the error message element. If Flask's Jinja2 template engine has correctly escaped the output, *<script>* becomes *<script>*, and will not appear in raw form.